

Ejercicio 1

1.-Redactar un documento que defina los objetivos de la arquitectura de base de datos para un sistema de gestión escolar.

Sistema: Gestión escolar (matrículas, alumnos, notas, asistencia, pagos, docentes).

Objetivos de la arquitectura de BD :

Guardar la información de forma ordenada

Que los datos de alumnos, cursos y notas estén bien organizados en tablas/estructuras claras.

Evitar errores y mantener datos correctos (integridad)

Que no existan “notas” sin alumno, ni cursos sin docente.

Usar reglas como llaves primarias y foráneas.

Permitir acceso rápido a la información (rendimiento)

Consultas frecuentes: reporte de notas, lista de alumnos por salón, asistencia por mes.

Usar índices en campos importantes (por ejemplo: DNI, código de alumno, código de curso).

Proteger la información (seguridad)

Roles: administrador, docente, secretaria, estudiante, padre.

Cada rol ve solo lo que necesita.

Asegurar disponibilidad

Que el sistema funcione casi siempre (por ejemplo, en época de matrículas).

Copias de seguridad (backups) y plan de recuperación.

Poder crecer (escalabilidad)

Si el colegio aumenta de 500 a 5000 alumnos, la BD debe seguir funcionando bien.

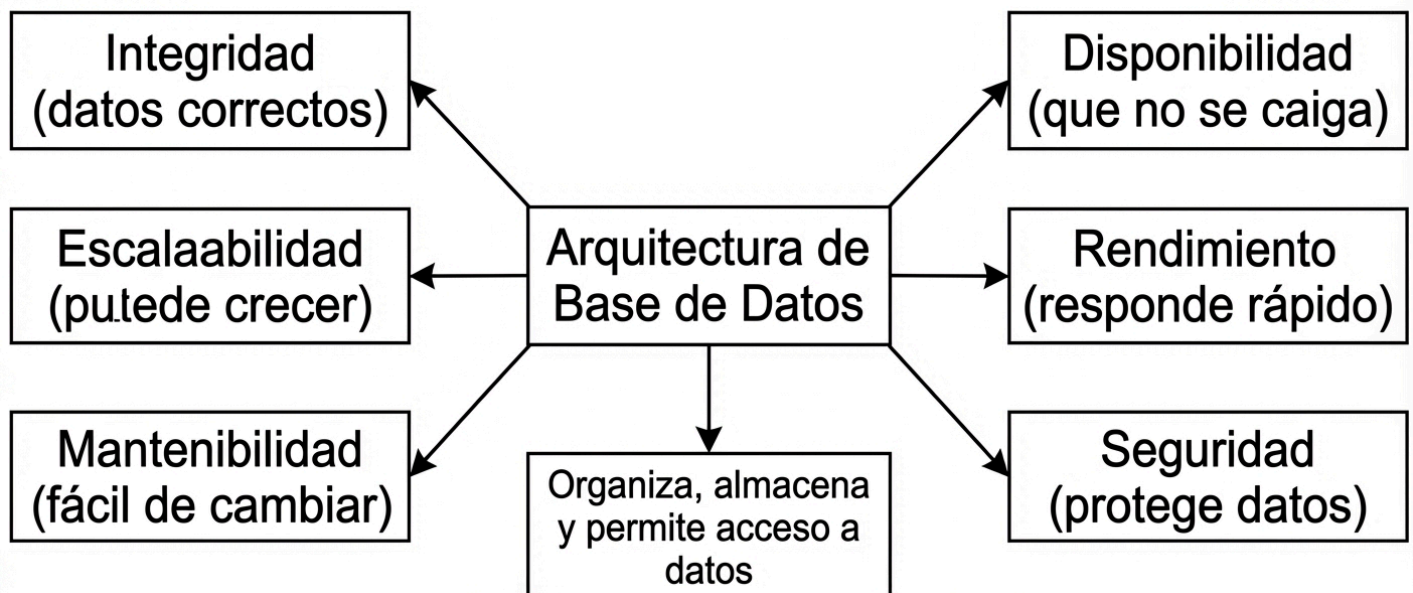
Facilitar mantenimiento y cambios

Que sea sencillo agregar un nuevo tipo de reporte o un nuevo módulo (biblioteca, comedor, etc.).

2.-Comparar los objetivos de la arquitectura de una empresa de retail vs una entidad bancaria.

Aspecto	Retail (tienda, e-commerce)	Banco
Prioridad principal	Velocidad y alto volumen de ventas	Seguridad y consistencia del dinero
Rendimiento	Muy alto (muchas consultas y compras)	Alto, pero con controles estrictos
Consistencia	A veces se tolera consistencia eventual (ej. stock)	Consistencia fuerte (saldo exacto siempre)
Seguridad	Alta (datos de clientes y pagos)	Muy alta (fraude, auditoría, regulación)
Disponibilidad	Alta (si cae el sistema, se pierden ventas)	Altísima (servicios críticos 24/7)
Auditoría y trazabilidad	Importante (ventas, devoluciones)	Crítica (cada movimiento debe quedar registrado)
Escalabilidad	Muy importante (picos: campañas, ofertas)	Importante (crecimiento de clientes y transacciones)

3.-Diseñar un mapa conceptual con los objetivos fundamentales de una arquitectura de base de datos.



Ejercicio 2:

1.- Identificar los componentes en el DBMS MySQL y explicar cómo se aplican en un caso real.

Componentes y cómo se ven en MySQL (caso real: sistema de ventas):

Almacenamiento

En MySQL se guarda en archivos del motor (por ejemplo InnoDB).

Caso: tablas **clientes**, **productos**, **ventas**.

Motor de consultas (Query Engine/Optimizer)

MySQL decide cómo ejecutar un **SELECT** de la forma más rápida.

Caso: "ver ventas del día" (usa índices para responder rápido).

Metadatos (Diccionario/Information Schema)

MySQL guarda información de tablas, columnas, tipos, índices.

Caso: revisar estructura con `INFORMATION_SCHEMA`.

Seguridad (usuarios y permisos)

MySQL maneja usuarios, roles y privilegios.

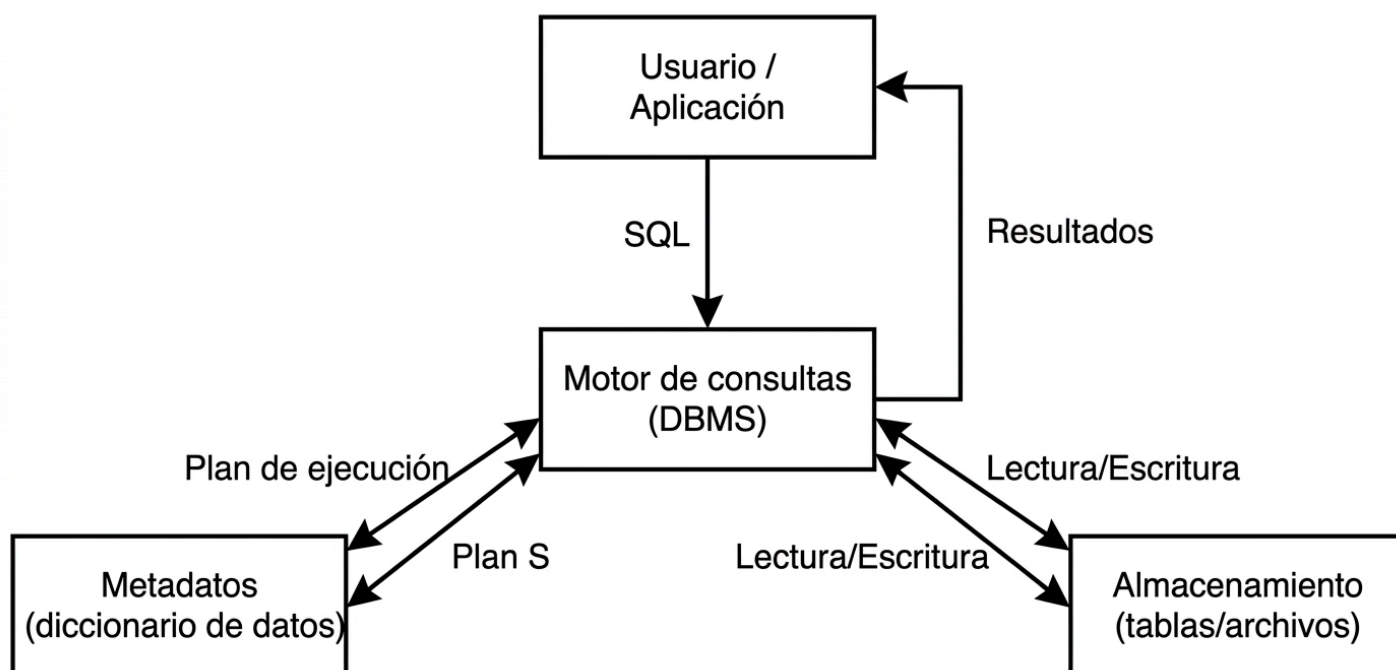
Caso: el usuario “cajero” solo puede insertar ventas; no puede borrar productos.

Interfaz / Conectividad

MySQL se conecta desde apps con drivers (JDBC, ODBC, Python, PHP).

Caso: una web de ventas se conecta con un driver y ejecuta consultas.

2.-Elaborar un diagrama sencillo de la interacción entre almacenamiento, motor de consultas y metadatos.



3.-Diseñar un esquema de seguridad de base de datos para un sistema de notas.

Actores: Administrador, Secretaria, Docente, Estudiante, Padre/Tutor.

Tablas típicas: `alumnos`, `cursos`, `matriculas`, `notas`, `asistencia`.

Reglas de seguridad (simple):

Administrador

Acceso total (crear usuarios, backups, cambios de estructura).

Secretaria

Puede: crear/editar alumnos, matrículas, cursos.

No puede: cambiar notas ya registradas.

Docente

Puede: registrar y editar notas solo de sus cursos.

Puede: ver alumnos de sus cursos.

No puede: ver notas de cursos de otros docentes.

Estudiante

Puede: ver solo sus notas y asistencia.

No puede: editar nada.

Padre/Tutor

Puede: ver notas/asistencia del estudiante asociado.

EJERCICIO 3:

1.- Investiga casos de uso de bases de datos centralizadas en instituciones educativas.

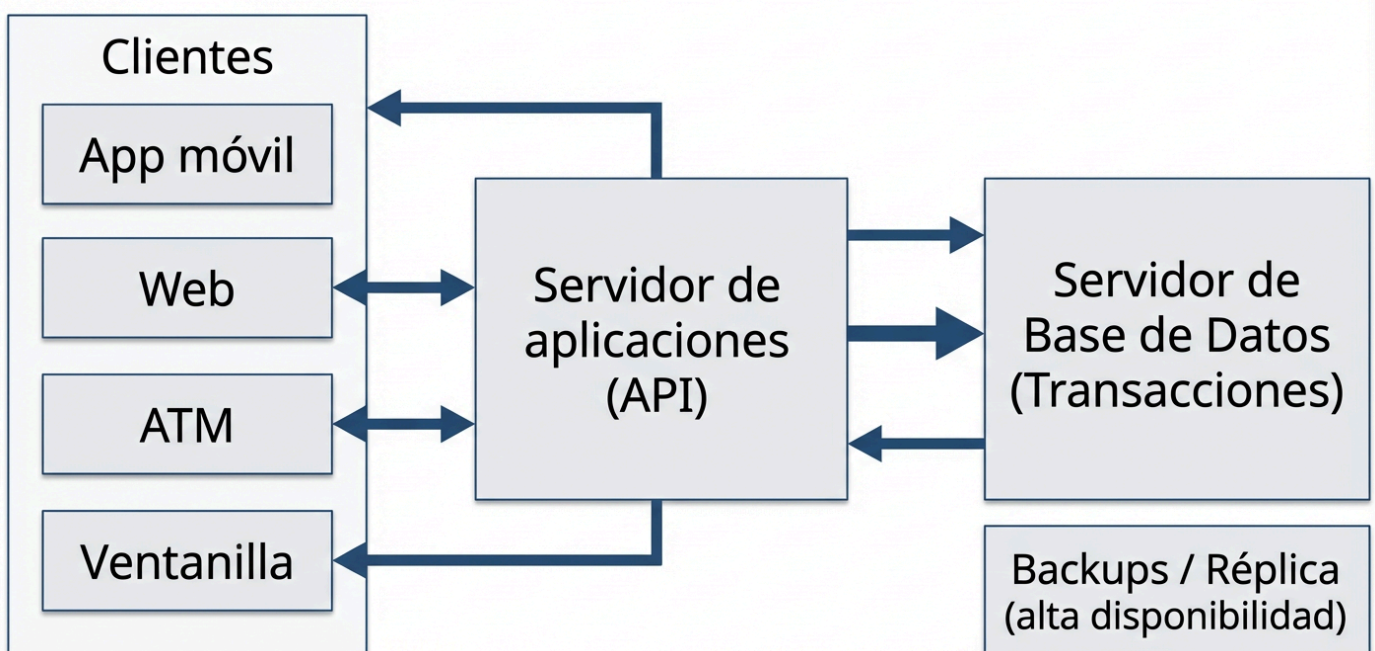
a) Colegio con una sola sede: servidor central para alumnos, notas, pagos.

b) Laboratorio de cómputo: todos los equipos usan la misma BD.

c) Sistema de biblioteca: préstamos y multas en una BD central.

Ventajas: fácil de administrar, menor costo. Limitación: si el servidor falla, todo se detiene.

2.- Diseña un diagrama de arquitectura cliente-servidor para un sistema bancario.



3.- Compara ventajas y limitaciones de una arquitectura en la nube vs distribuida.

Aspecto	Nube (Cloud)	Distribuida (varios nodos)
Ventaja principal	Escala rápida y pago por uso	Alta disponibilidad y tolerancia a fallos
Implementación	Más rápida (servicios administrados)	Más compleja (coordinación, replicación)
Costo	Flexible, puede subir si no se controla	Puede ser alto por infraestructura y mantenimiento
Riesgos	Dependencia del proveedor (vendor lock-in)	Complejidad de consistencia y red
Ideal para	Startups, apps con demanda variable	Sistemas críticos y con alta tolerancia a fallos

EJERCICIO 4:

1.-Elaborar una tabla que relacione arquitectura con requisito de rendimiento y disponibilidad.

Arquitectura	Rendimiento	Disponibilidad	Comentario
Centralizada	Media	Baja-Media	Un solo servidor, punto único de falla
Cliente-servidor	Media-Alta	Media	Mejora organización, depende del servidor BD
Distribuida	Alta (bien diseñada)	Alta	Requiere coordinación y buen diseño
Nube	Alta	Alta	Escala fácil, depende del proveedor
Federada	Media	Media	Integra varias fuentes, consultas pueden ser más lentas
Orientada a servicios (SOA)	Media	Media-Alta	Flexible por módulos, depende de servicios

2.-Diseñar un informe donde se justifique la elección de una arquitectura para un sistema hospitalario.

Contexto: Hospital con áreas (emergencia, laboratorio, farmacia, consultorios). Muchos usuarios simultáneos.

Requisitos principales:

Alta disponibilidad (no puede detenerse).

Seguridad (datos médicos son sensibles).

Rendimiento (consultas rápidas, sobre todo en emergencias).

Auditoría (quién vio o cambió un dato).

Arquitectura recomendada:

Cliente-servidor + base de datos central con réplicas (o servicio en nube administrado con alta disponibilidad).

Justificación:

Central para mantener un historial único del paciente.

Réplicas para que si un servidor cae, el sistema siga.

Roles claros (médico, enfermera, admisión, farmacia).

Backups automáticos.

Conclusión: Esta arquitectura equilibra control (consistencia médica) con disponibilidad (servicio continuo).

3.-Resolver un caso en el que se priorice consistencia fuerte y explicar la elección de arquitectura.

Caso : Transferencias bancarias entre cuentas (A -> B).

Por qué se prioriza consistencia fuerte:

El dinero no puede “aproximarse”. El saldo debe ser exacto en todo momento.

No se permiten errores de sincronización. No puede pasar que una cuenta se descuente y la otra no se sume (o al revés).

Evitar duplicación o pérdida de dinero. En un sistema con consistencia débil, por retrasos entre servidores, podrían verse saldos incorrectos temporalmente. En banca eso no es aceptable.

Arquitectura elegida :

Base de datos transaccional con consistencia fuerte (ACID), por ejemplo un RDBMS (PostgreSQL, Oracle, SQL Server) configurado para transacciones.

Operación en una sola transacción: el sistema hace “debit” y “credit” en el mismo proceso; si algo falla, se hace rollback y no queda el sistema a medias.

Clúster/replicación con control estricto: si se usa más de un nodo, se elige una configuración que mantenga consistencia (por ejemplo, un nodo líder para escrituras o replicación sincrónica en lo crítico), aunque sea más costoso o un poco más lento.

Conclusión:

En transferencias bancarias se elige una arquitectura transaccional (ACID) porque se prioriza que los datos sean correctos siempre, incluso si eso reduce un poco la velocidad o complica la escalabilidad.

EJERCICIO 5:

1.-Diseñar un modelo conceptual para una biblioteca y transformarlo a modelo lógico y físico.

A) Modelo conceptual (idea, sin tablas):

Entidades: Libro, Autor, Usuario, Préstamo.

Relaciones:

Un Autor escribe uno o varios Libros.

Un Usuario realiza varios Préstamos.

Un Préstamo incluye un Libro y un Usuario, con fechas.

B) Modelo lógico (tablas y relaciones):

autor(id_autor, nombre)

libro(id_libro, titulo, anio, id_autor)

usuario(id_usuario, nombres, dni)

prestamo(id_prestamo, id_usuario, id_libro, fecha_prestamo, fecha_devolucion)

C) Modelo físico (ejemplo en MySQL, tipos):

autor

id_autor INT AUTO_INCREMENT PRIMARY KEY

nombre VARCHAR(100) NOT NULL

libro

id_libro INT AUTO_INCREMENT PRIMARY KEY

titulo VARCHAR(200) NOT NULL

anio YEAR

id_autor INT NOT NULL

FOREIGN KEY (id_autor) REFERENCES autor(id_autor)

usuario

id_usuario INT AUTO_INCREMENT PRIMARY KEY

nombres VARCHAR(150) NOT NULL

dni VARCHAR(12) UNIQUE

prestamo

id_prestamo INT AUTO_INCREMENT PRIMARY KEY

id_usuario INT NOT NULL

id_libro INT NOT NULL

fecha_prestamo DATE NOT NULL

fecha_devolucion DATE NULL

FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)

FOREIGN KEY (id_libro) REFERENCES libro(id_libro)

2.-Explicar en un informe las diferencias entre cada nivel de abstracción en un sistema hospitalario.

Informe :

Nivel conceptual (qué existe)

Define entidades: Paciente, Médico, Cita, Historia clínica, Receta.

No habla de tablas ni tipos de datos.

Nivel lógico (cómo se organiza en BD)

Se convierte en tablas y relaciones.

Ejemplo: **paciente**, **medico**, **cita**, **receta** y sus llaves foráneas.

Nivel físico (cómo se guarda realmente)

Tipos de datos, índices, particiones, almacenamiento.

Ejemplo: índices para buscar rápido por DNI o por número de historia.

Conclusión:

Conceptual: visión del negocio.

Lógico: diseño para BD.

Físico: implementación técnica.

3.-Caso práctico de migración de un modelo conceptual a físico en MySQL (pasos).

Definir entidades y relaciones

Ejemplo: Cliente, Producto, Venta.

Crear tablas (lógico)

cliente, producto, venta, detalle_venta.

Elegir tipos de datos (físico)

IDs como **INT**, nombres como **VARCHAR**, fechas como **DATE/DATETIME**.

Crear llaves primarias y foráneas

Para asegurar integridad.

Agregar índices

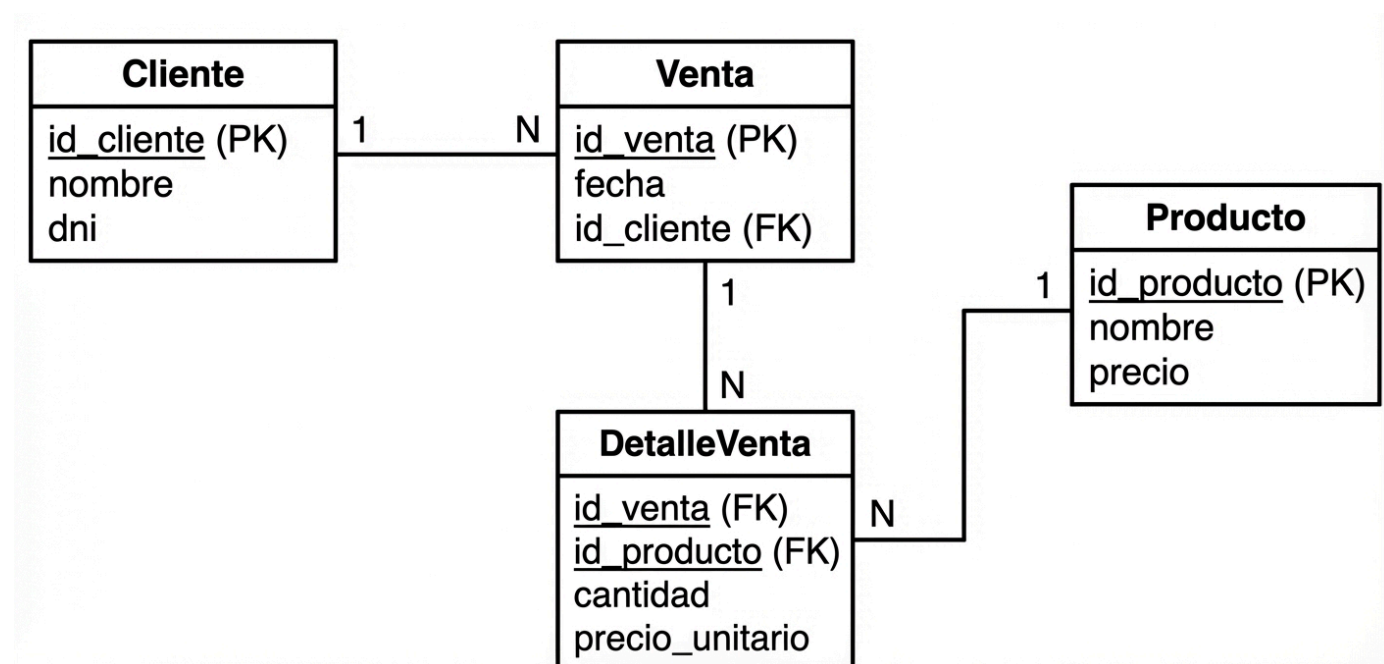
Por ejemplo: **venta(fecha)** y **cliente(dni)**.

Probar con datos y consultas reales

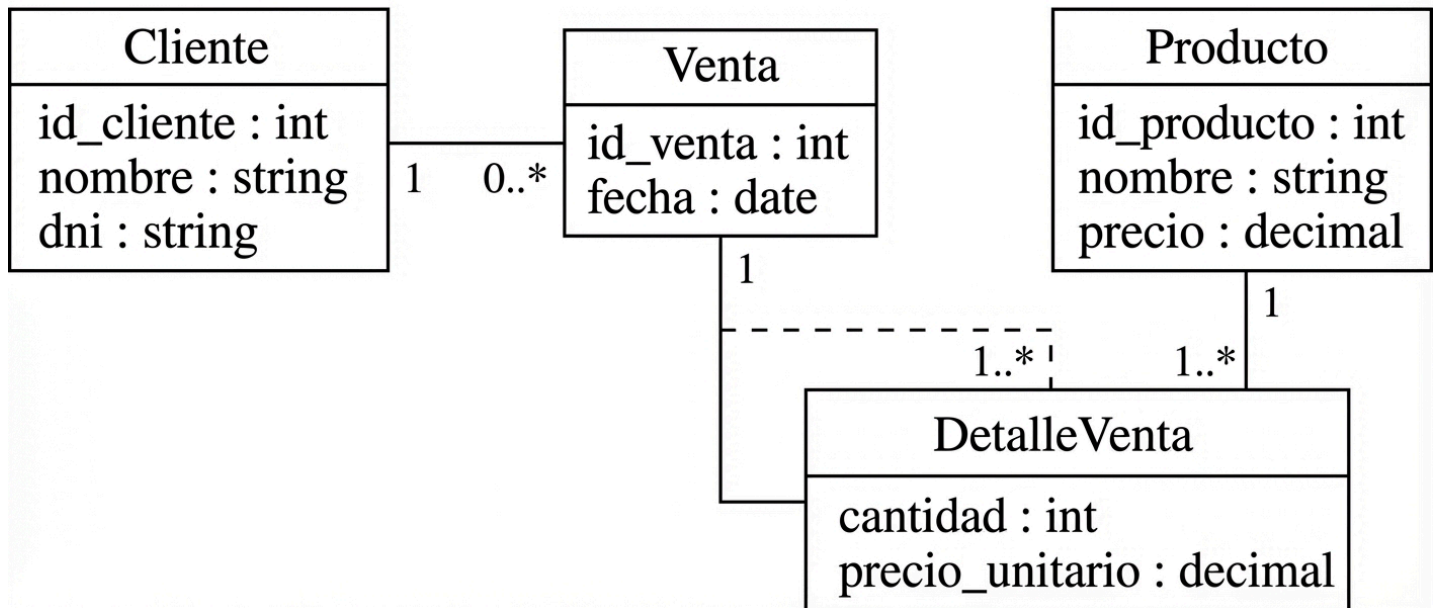
Reporte diario de ventas, productos más vendidos.

EJERCICIO 6:

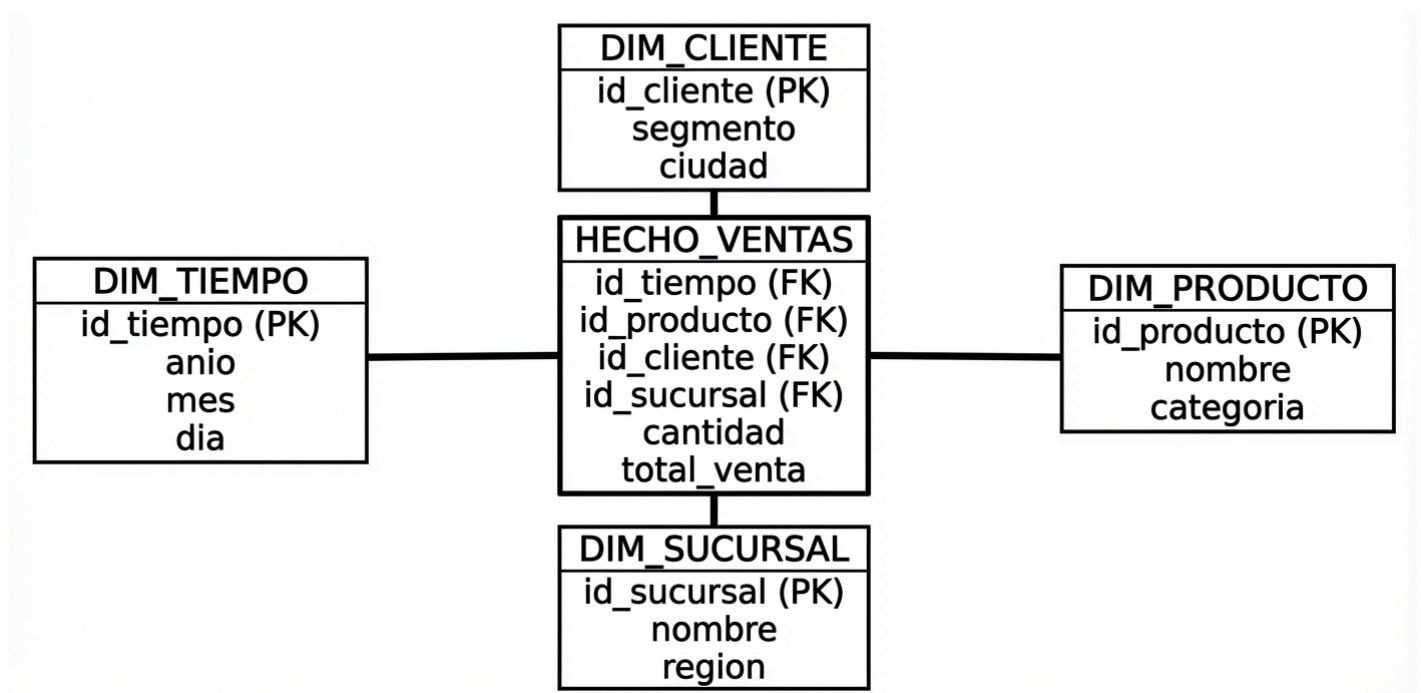
1.- Elabora un diagrama ER para un sistema de ventas.



2.- Representa el mismo diagrama en UML y compara diferencias con el ER.



3.- Diseña un modelo estrella para analizar ventas anuales.



EJERCICIO 7:

1.-Normalizar una tabla de pedidos con datos redundantes hasta 3FN (ejemplo).

Tabla inicial :

```
pedido(id_pedido, fecha, dni_cliente, nombre_cliente, direccion_cliente,
id_producto, nombre_producto, precio, cantidad)
```

Problemas:

Se repite `nombre_cliente`, `direccion_cliente` en cada fila.

Se repite `nombre_producto`, `precio` en cada fila.

A) 1FN (valores atómicos):

Asegurar que no haya listas dentro de una celda (ok en el ejemplo).

B) 2FN (depende de toda la clave):

Separar datos que dependen solo del cliente o producto.

C) 3FN (sin dependencias transitivas):

Tablas normalizadas:

```
cliente(dni_cliente PK, nombre, direccion)
```

```
producto(id_producto PK, nombre, precio)
```

```
pedido(id_pedido PK, fecha, dni_cliente FK)
```

```
detalle_pedido(id_pedido FK, id_producto FK, cantidad, precio_unitario)
```

2.-Identificar problemas de redundancia en una base de datos académica y rediseñarla (ejemplo).

Problema típico: tabla `matricula` guarda también nombre del alumno, nombre del curso y nombre del docente.

Riesgos:

Si cambia el nombre del docente, hay que cambiarlo en muchas filas.

Se crean datos inconsistentes.

Rediseño (simple):

```
alumno(id_alumno, nombre, dni)
```

```
docente(id_docente, nombre)
```

```
curso(id_curso, nombre, id_docente)
```

```
matricula(id_matricula, id_alumno, id_curso, periodo)
```

```
nota(id_nota, id_matricula, nota1, nota2, promedio)
```

3.-Cuadro comparativo: ventajas y desventajas de la normalización.

Normalización	Ventajas	Desventajas
Aplicarla	- Menos redundancia - Menos errores - Datos más consistentes - Más fácil mantener	- Más tablas - Consultas con más JOIN - Puede ser más lenta para reportes complejos

EJERCICIO 8:

1.-Proponer un caso donde la desnormalización mejora el rendimiento de un sistema de reportes.

Caso: Reporte diario de ventas por sucursal y categoría, consultado muchas veces.

Desnormalización propuesta:

Crear una tabla de resumen:

```
resumen_ventas_diario(fecha, sucursal, categoria, total_ventas,  
total_cantidad)
```

Por qué mejora:

En vez de hacer JOIN entre muchas tablas cada vez, se consulta una tabla ya lista.

2.-Diseñar una vista materializada en Oracle o PostgreSQL para un sistema de ventas.

Objetivo: Total de ventas por mes y producto.

PostgreSQL (simulación con tabla resumen, porque PostgreSQL no trae materialized view en todas las configuraciones antiguas, pero en versiones modernas sí existe):

```
CREATE MATERIALIZED VIEW mv_ventas_mes_producto AS  
  
SELECT  
  
    date_trunc('month', v.fecha) AS mes,  
  
    dv.id_producto,  
  
    SUM(dv.cantidad) AS total_cantidad,  
  
    SUM(dv.cantidad * dv.precio_unitario) AS total_venta  
  
FROM venta v  
  
JOIN detalle_venta dv ON dv.id_venta = v.id_venta  
  
GROUP BY 1, 2;
```

Actualización (refresco):

```
REFRESH MATERIALIZED VIEW mv_ventas_mes_producto;
```

3.-Justificar cuándo es conveniente materializar consultas en un sistema de BI (texto).

Conviene materializar cuando:

Hay reportes pesados que se consultan muchas veces.

Los datos no cambian cada minuto (por ejemplo, se actualizan cada noche).

Se necesita respuesta rápida para gerencia.

No conviene cuando:

Los datos cambian todo el tiempo y se necesita información en tiempo real.

No se quiere el costo extra de almacenamiento y mantenimiento.